

DATABRICKS AND SNOWFLAKE | CAPSTONE
PROJECT 2026

Job-posting-skill-demand

A retail sales lakehouse pipeline built on Databricks and served through Snowflake

NAME	SRIDEEP ADAK
ROLL NUMBER	23051631
BATCH	DATABRICKS & SNOWFLAKE 2026
SUBMITTED TO	Kanthi Kiran Sirra
DATE	26 th September, 2026

1. Problem Statement

Technology teams need a clear view of which skills occur most often together and how their representation changes over time. Raw job-posting data is unsuitable for analysis because it contains duplicate records, invalid dates, missing skills and inconsistent skill spellings. Analysts therefore need a reproducible pipeline that cleans the source data and produces consistent analytical measures.

Item	Project Definition
Domain	Technology skill-demand analytics
Users	Data analysts, data engineers, trainers and project assessors
Problem	Job-posting records contain duplicates, invalid values and inconsistent skill names.
Dataset	Synthetic job postings generated for the capstone scope
Period	January 2024 to June 2025
Success criteria	Validated Bronze/Silver/Gold processing, reproducible exports, successful Snowflake analysis and verified dashboard metrics.

The project deliberately uses synthetic data. Its findings describe the generated dataset and are not presented as measurements or forecasts of the live labour market.

2. Objectives and Scope

- Generate and ingest the synthetic job-posting source data.
- Clean, validate and standardize postings and skill names using the Bronze, Silver and Gold approach.
- Produce monthly skill-demand, quarter-on-quarter and skill-pair analytical outputs.
- Load the curated Gold-layer CSV exports into Snowflake and verify the analytical results.
- Provide a lightweight static dashboard containing the verified project metrics.

In Scope	Out of Scope
18-month synthetic job-posting analysis	Live web scraping
Databricks Bronze/Silver/Gold pipeline	Real-time labour-market forecasting
Snowflake loading and SQL verification	Unattended Databricks-to-Snowflake integration
Static dashboard and reproducibility checks	Claims about the actual labour market

Table 1. Project scope.

3. Solution and Features

The solution combines Databricks for transformation and scheduled processing with Snowflake for the analytical serving layer. The pipeline retains raw provenance in Bronze, cleans and standardizes data in Silver, and creates business-ready Gold metrics.

- Bronze, Silver and Gold layers with preserved raw provenance and explicit rejection reasons.
- Monthly top-ten skill demand and skill-share analysis.
- Quarter-on-quarter share-change analysis.
- Skill-pair co-occurrence using pair counts, confidence and lift.
- Deterministic exports and SHA-256 fingerprints for reproducibility.
- Snowflake repeated-load verification to prevent duplicate ingestion.
- A static dashboard that presents the verified project outputs.

4. Tech Stack

Technology	Purpose in this project	Why it was chosen
Python	Synthetic data generation and validation logic	Simple, reproducible scripting.
PySpark / Spark SQL	Cleaning, joins, skill normalization and aggregations	Distributed data processing and SQL support.
Databricks	Cloud execution, Bronze/Silver/Gold tables and scheduling	Managed Spark environment and notebook workflow.
Snowflake	Stage, COPY INTO, analytical SQL and verification	Cloud warehouse for structured analytical queries.
CSV and SHA-256	Portable curated handoff and result fingerprints	Simple transfer and reproducibility checks.
HTML, CSS and JavaScript	Static project dashboard	Lightweight browser-based presentation.

Table 2. Technology stack.

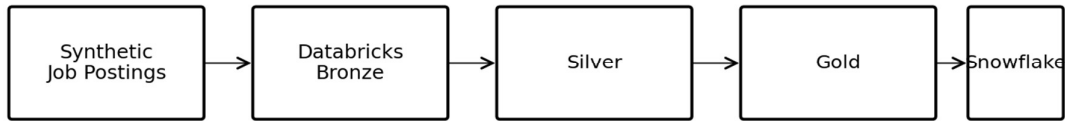
5. Data Design and Layers

Layer	Main Content	Purpose
Bronze	Raw job postings and company information	Preserve input values, source provenance and auditability.
Silver	Validated posting-skill records	Reject invalid records, split skills and standardize names.
Gold	Skill-month and pair-month metrics	Provide ranked demand and companion-skill analytics.

Table 3. Medallion-style data design.

The Silver layer records rejected rows and the reason for rejection rather than silently removing them. This allows the pipeline to be audited and makes data-quality outcomes measurable.

6. System Architecture



Validation: local Spark → Databricks → CSV reconciliation → Snowflake SQL checks

Figure 1. Implemented data path and validation boundary.

Raw synthetic postings are transformed in Databricks. Bronze retains source values and provenance. Silver removes duplicates and invalid records before splitting and standardizing skills. Gold contains monthly skill measures and co-occurrence measures. Curated CSV exports are placed in a Snowflake named stage and loaded with COPY INTO before analytical SQL and full-result verification are run. The scheduled component is Databricks. The CSV transfer to Snowflake was tested manually. End-to-end validation covered local Spark, Databricks execution, cloud CSV reconciliation and Snowflake SQL checks.

7. Implementation Highlights

The implementation eliminates duplicates before exploding the skill list, preventing duplicated postings from multiplying downstream measures. It splits comma- and semicolon-separated skill lists, trims whitespace, lowercases values and maps aliases to canonical names. Skill share uses accepted postings in the month as its denominator.

Control	Implemented behavior
Input rejection	Duplicate, missing-skill, unknown-company and invalid-date records receive explicit reasons.
Determinism	A fixed generated scope, ordered results and SHA-256 result fingerprints make reruns comparable.
Databricks compatibility	The date offset is explicitly cast to INT for Spark 4; the original source is preserved.
Query correctness	Pair aggregation occurs before ranking; quarterly division uses explicit DOUBLE precision.

Table 4. Implementation decisions that protect metric correctness.

8. Testing and Results

Test Case	Expected Result	Actual Result
Local Spark pipeline	Repeatable outputs and validations	Two full runs; all 31 checks passed; identical exports.
Independent reconciliation	Gold/Silver agree with oracle	All posting-skill, monthly, pair and rejection values agree.
Databricks pipeline	Tables and notebook expectations complete	All 26 expectations passed.
Scheduled execution	Scheduled cloud run completes	Run 365017355708433 succeeded in 2 minutes 42 seconds.
CSV transfer	Cloud exports match local evidence	Every downloaded row agrees; 1e-12 tolerance for floating measures.
Snowflake first load	Curated rows load without errors	360, 6,660 and 227,624 rows loaded; zero errors.
Repeated Snowflake load	Same files add no duplicate data	All three files LOAD_SKIPPED with zero rows added.
Snowflake analysis	Results match expected outputs	14 checks passed; 0 failed. Q1 180, Q2 120 and Q3 60 rows.

9. Evaluation and Metrics

The evaluation does not depend only on whether the code runs. The project uses row counts, rejection counts, deterministic fingerprints, full-row reconciliation, repeated-load behaviour and analytical-query result sets.

Metric	Verified Value
Input postings	55,350
Accepted postings	51,840
Canonical skills	20 from 57 raw variants
Monthly Gold rows	360
Pair Gold rows	6,660
Silver posting-skill rows	227,624
Snowflake final checks	14 passed / 0 failed
Scheduled run duration	2 minutes 42 seconds

Table 6. Key evaluation metrics.

10. Screenshots



Figure 10.1: Dashboard

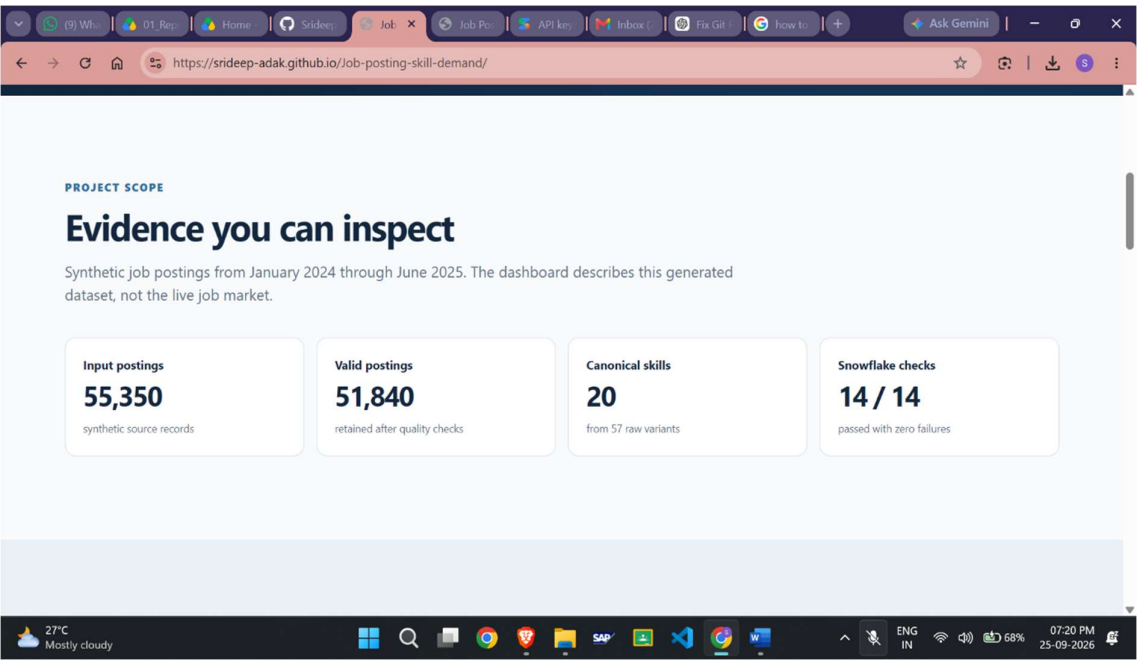


Figure 10.2: Project Scope



Figure 10.3: Monthly Skill Demand



Figure 10.4: Top skills in the final month

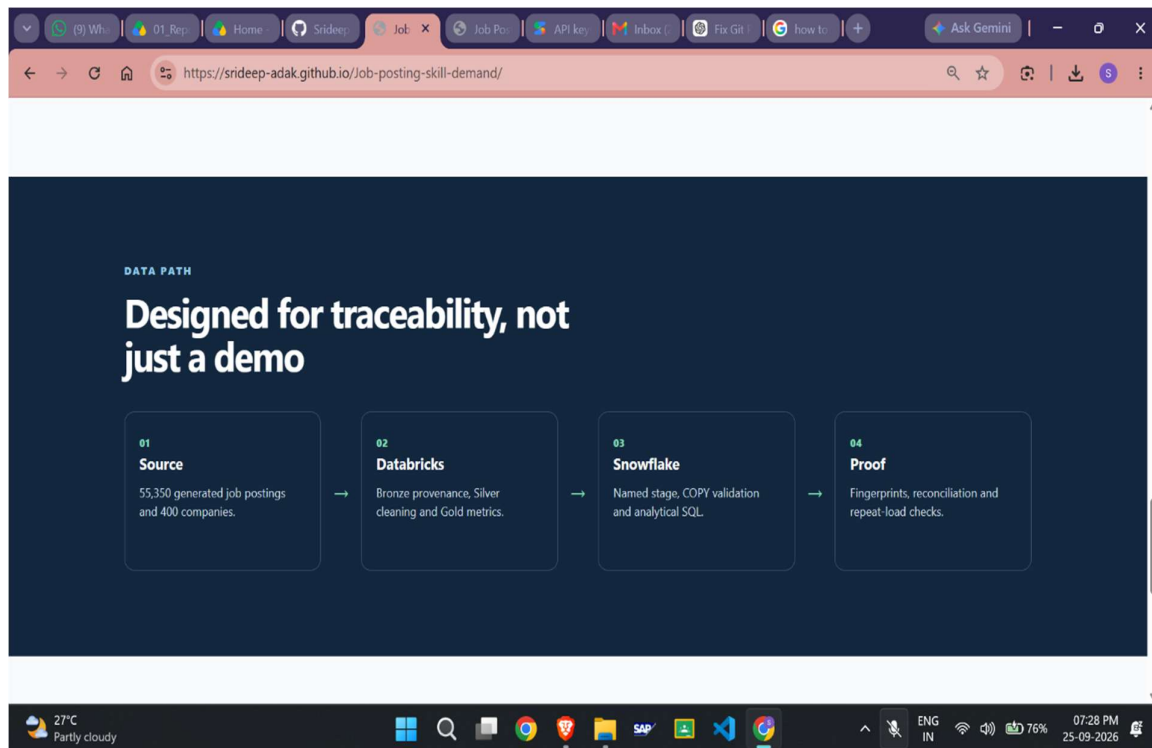


Figure 10.5: Data Path

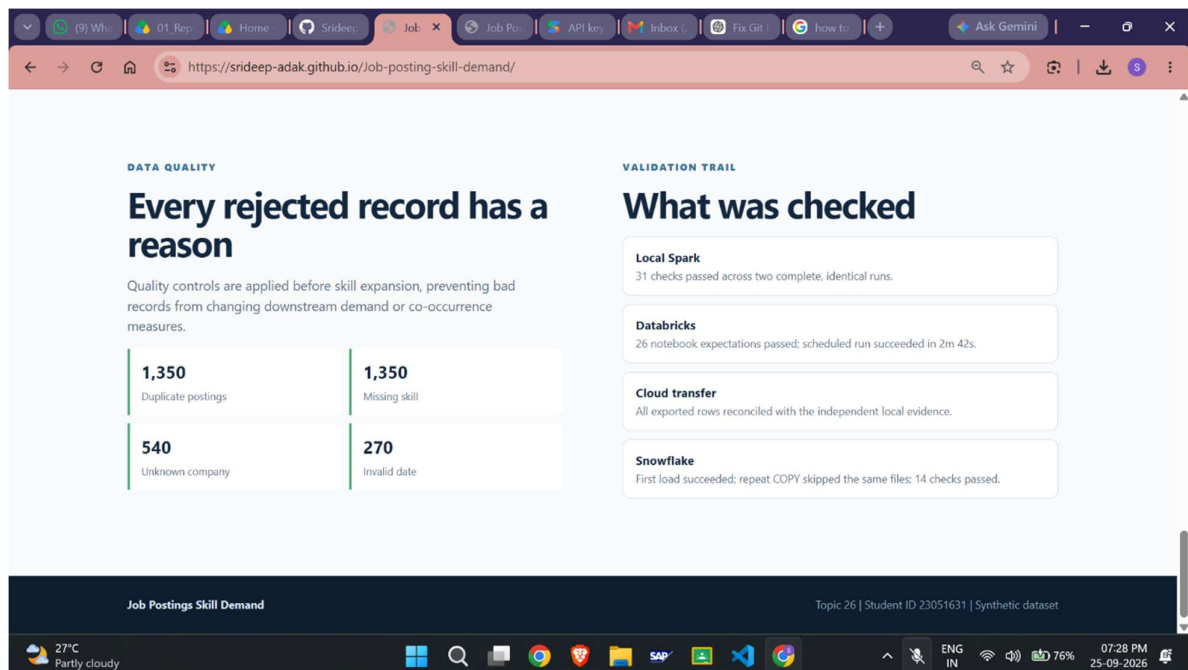


Figure 10.6: Data Quality & Validation Trail

11. Challenges and Learnings

- Spark 4 requires an integer date offset. Preserving the original source alongside the compatible execution path kept the change auditable.
- Analytical queries required engine-aware SQL: aggregate first, then rank, and make decimal precision explicit for cross-engine comparison.
- A rerun-safe Snowflake load must be tested rather than assumed. The repeated COPY check confirmed that the same files were not loaded twice.
- Cloud results were reconciled against local outputs so an individual success message was not treated as proof of analytical correctness.

12. Future Improvements

Add controlled snapshot replacement or file-version management for future ingestion cycles.

- Add an orchestration layer when unattended Databricks-to-Snowflake integration is required by a future scope.
- Use a real, ethically sourced dataset with documented permissions if the project later needs market-level conclusions.
- Add filters for city, sector and company size to extend the dashboard analysis.

13. Conclusion

Add controlled snapshot replacement or file-version management for future ingestion cycles.

- Add an orchestration layer when unattended Databricks-to-Snowflake integration is required by a future scope.
- Use a real, ethically sourced dataset with documented permissions if the project later needs market-level conclusions.
- Add filters for city, sector and company size to extend the dashboard analysis.

14. References and Links

Reference	Details
Capstone Project Briefs	Topic 26, pages 104–107, supplied course document.
Databricks documentation	input_file_name and file-provenance guidance used for the pipeline.
Snowflake documentation	COPY INTO and data-loading guidance used for the warehouse stage.
Project repository	Srideep-Adak/Job-posting-skill-demand; source code, SQL, notebook, dashboard and evidence.
Dashboard	GitHub Pages deployment from the repository's docs directory.